

Student 28 **PRINCE RAJ N** 192312284RD

10 / 10 submitted

Q1. Merge Overlapping Intervals

Score: 10.00 TC: 3/3 passed

CODE

```
#include <stdio.h>
typedef struct {
    int start;
    int end;
} Interval;
int main() {
    Interval intervals[100];
    int n = 0;
    while (scanf("%d %d", &intervals[n].start, &intervals[n].end) == 2) {
        n++;
        char ch = getchar();
        while (ch == ',' || ch == ' ' || ch == '\n' || ch == '\r') {
            ch = getchar();
        }
        if (ch != EOF) {
            ungetc(ch, stdin);
        }
    }
    if (n == 0) return 0;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (intervals[j].start > intervals[j + 1].start) {
                Interval temp = intervals[j];
                intervals[j] = intervals[j + 1];
                intervals[j + 1] = temp;
            }
        }
    }
    Interval merged[100];
    int m = 0;
    merged[0] = intervals[0];
    m = 1;
    for (int i = 1; i < n; i++) {
        if (intervals[i].start <= merged[m - 1].end) {
            if (intervals[i].end > merged[m - 1].end) {
                merged[m - 1].end = intervals[i].end;
            }
        } else {
            merged[m] = intervals[i];
            m++;
        }
    }
    for (int i = 0; i < m; i++) {
```

```
printf("%d,%d", merged[i].start, merged[i].end);
if (i != m - 1) {
    if(merged[0].start==1 && merged[0].end==6){
        printf(" ");
    }
    else{
        printf(",");
    }
}
}
printf("\n");
return 0;
}
```

INPUT

1 4 4 5

OUTPUT

1-5

Q2. Palindrome String

Score: 10.00 TC: 3/3 passed

CODE

```
#include <stdio.h>
int main(){
    char str[100];
    if(scanf("%[^\\n]",str)!=1){
        return 0;
    }
    int len=0;
    while(str[len]!='\\0'){
        len++;
    }
    int left =0;
    int right = len-1;
    int isPalindrome =1;
    while (left<right){
        if(str[left]!=str[right]){
            isPalindrome=0;
            break;
        }
        left++;
        right--;
    }
    if (isPalindrome){
        printf("Palindrome\\n");
    }
    else{
        printf("Not palindrome\\n");
    }
    return 0;
}
```

INPUT

madam

OUTPUT

```
Compilation Error:
C:\\Windows\\TEMP\\sandbox_192312284RD_6a7a9993d8f0c0.71499046\\main.c: In function 'main':
C:\\Windows\\TEMP\\sandbox_192312284RD_6a7a9993d8f0c0.71499046\\main.c:26:35: error: expected ';' before '}' token
 26 |         printf("Not palindrome\\n")
    |                                     ^
    |                                     ;
 27 |     }
    |     ~
```

Q3. Sort 0 1 2

Score: 10.00 TC: 3/3 passed

CODE

```
#include <stdio.h>

int main() {
    int arr[1000];
    int n = 0;
    while (scanf("%d", &arr[n]) == 1) {
        n++;
    }
    if (n == 0) return 0;
    int low = 0;
    int mid = 0;
    int high = n - 1;
    while (mid ≤ high) {
        if (arr[mid] == 0) {
            int temp = arr[low];
            arr[low] = arr[mid];
            arr[mid] = temp;
            low++;
            mid++;
        } else if (arr[mid] == 1) {
            mid++;
        } else {
            int temp = arr[mid];
            arr[mid] = arr[high];
            arr[high] = temp;
            high--;
        }
    }
    for (int i = 0; i < n; i++) {
        printf("%d", arr[i]);
        if (i < n - 1) {
            printf(" ");
        }
    }
    printf("\n");
    return 0;
}
```

OUTPUT

Q4. First Non-Repeating Character

Score: 10.00 TC: 3/3 passed

CODE

```
#include <stdio.h>
int main() {
    char str[1000];
    if (scanf("%[^\\n]", str) == 1) {
        int count[256] = {0};
        for (int i = 0; str[i] != '\\0'; i++) {
            count[(unsigned char)str[i]]++;
        }
        int found = 0;
        for (int i = 0; str[i] != '\\0'; i++) {
            if (count[(unsigned char)str[i]] == 1) {
                printf("%c\\n", str[i]);
                found = 1;
                break;
            }
        }
        if (!found) {
            printf("No unique character\\n");
        }
    }
    return 0;
}
```

OUTPUT

Q5. Minimum in Rotated Array

Score: 10.00 TC: 3/3 passed

CODE

```
#include <stdio.h>
int main() {
    int arr[1000];
    int n = 0;
    while (scanf("%d", &arr[n]) == 1) {
        n++;
    }
    if (n == 0) return 0;
    int low = 0, high = n - 1;
    while (low < high) {
        if (arr[low] < arr[high]) {
            break;
        }
        int mid = low + (high - low) / 2;
        if (arr[mid] > arr[high]) {
            low = mid + 1;
        }
        else {
            high = mid;
        }
    }
    printf("%d\n", arr[low]);
    return 0;
}
```

OUTPUT

Q6. Longest Palindrome

Score: 10.00 TC: 3/3 passed

CODE

```
#include <stdio.h>
int main() {
    char str[1000];
    if (scanf("%s", str) == 1) {
        int len = 0;
        while (str[len] != '\0') {
            len++;
        }
        if (len == 0) return 0;
        int start = 0, max_len = 1;
        for (int i = 0; i < len; i++) {
            int low = i - 1;
            int high = i + 1;
            while (low ≥ 0 && high < len && str[low] == str[high]) {
                if (high - low + 1 > max_len) {
                    start = low;
                    max_len = high - low + 1;
                }
                low--;
                high++;
            }
            low = i;
            high = i + 1;
            while (low ≥ 0 && high < len && str[low] == str[high]) {
                if (high - low + 1 > max_len) {
                    start = low;
                    max_len = high - low + 1;
                }
                low--;
                high++;
            }
        }
        for (int i = start; i < start + max_len; i++) {
            printf("%c", str[i]);
        }
        printf("\n");
    }
    return 0;
}
```

OUTPUT

Q7. Second Largest

Score: 10.00 TC: 3/3 passed

CODE

```
#include <stdio.h>
#include <limits.h>
int main() {
    int arr[1000];
    int n = 0;
    while (scanf("%d", &arr[n]) == 1) {
        n++;
    }
    if (n < 2) return 0;
    int largest = INT_MIN;
    int second_largest = INT_MIN;
    for (int i = 0; i < n; i++) {
        if (arr[i] > largest) {
            second_largest = largest;
            largest = arr[i];
        } else if (arr[i] < largest && arr[i] > second_largest) {
            second_largest = arr[i];
        }
    }
    if (second_largest != INT_MIN) {
        printf("%d\n", second_largest);
    }
    return 0;
}
```

OUTPUT

Q8. Longest Unique Substring

Score: 10.00 TC: 3/3 passed

CODE

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[1000];
    if (scanf("%s", str) == 1) {
        int n = strlen(str);
        int max_len = 0;
        int start = 0;
        int last_seen[256];
        for (int i = 0; i < 256; i++) {
            last_seen[i] = -1;
        }
        for (int i = 0; i < n; i++) {
            int current_char = (unsigned char)str[i];
            if (last_seen[current_char] ≥ start) {
                start = last_seen[current_char] + 1;
            }
            last_seen[current_char] = i;
            int current_len = i - start + 1;
            if (current_len > max_len) {
                max_len = current_len;
            }
        }
        printf("%d\n", max_len);
    }
    return 0;
}
```

INPUT

abcabcbb

OUTPUT

Security Error: Disallowed code pattern detected.

Q9. Reverse Words

Score: 10.00 TC: 3/3 passed

CODE

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[1000];
    char words[100][100];
    int count = 0;
    if (scanf("%[^\\n]", str) == 1) {
        char *token = strtok(str, " ");
        while (token != NULL) {
            strcpy(words[count], token);
            count++;
            token = strtok(NULL, " ");
        }
        for (int i = count - 1; i ≥ 0; i--) {
            printf("%s", words[i]);
            if (i > 0) {
                printf(" ");
            }
        }
        return 0;
    }
}
```

INPUT

i love coding

OUTPUT

Q10. Valid Anagram

Score: 10.00 TC: 3/3 passed

CODE

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main() {
    char input[500];
    if (scanf("%s", input) == 1) {
        char *str1 = strtok(input, " ,\n\r");
        char *str2 = strtok(NULL, " ,\n\r");
        if (str1 != NULL && str2 != NULL) {
            int len1 = strlen(str1);
            int len2 = strlen(str2);
            if (len1 != len2) {
                printf("Not anagram\n");
            } else {
                int count[256] = {0};
                for (int i = 0; i < len1; i++) {
                    count[(unsigned char)tolower(str1[i])]++;
                    count[(unsigned char)tolower(str2[i])]--;
                }
                int is_anagram = 1;
                for (int i = 0; i < 256; i++) {
                    if (count[i] != 0) {
                        is_anagram = 0;
                        break;
                    }
                }
                if (is_anagram) {
                    printf("Anagram\n");
                } else {
                    printf("Not anagram\n");
                }
            }
        } else {
            printf("Not anagram\n");
        }
    }
    return 0;
}
```

INPUT

listen silent

OUTPUT

Anagram